

# The Project, Explained End to End

A Conversational Platform for Financial Literacy and Management using a Multi-Agent LLM Orchestrator

Written for the team. 28 September 2026.

**How to read this.** Parts 1 to 4 build on each other, so read them in order. Parts 5 to 7 are reference: come back to them when you need a detail. Parts 8 to 10 explain what we ran, how the numbers were made, and what the paper is. Part 11 is a glossary: every technical word in this document is explained there in plain language, including the ones that sound obvious but are easy to mix up.

# 1. The project in one page

**The problem.** Many young Indians start earning with very little financial knowledge. A national survey in 2019 found that only 27 percent of Indian adults were financially literate. Tax is where this hurts first: from the first salary, a person faces two tax regimes, deductions, house rent allowance, and for business owners and freelancers, GST and tax deducted by clients.

Two kinds of help exist today, and each fails in a different way. **Tax calculators** give correct numbers but assume you already understand the words and what to type in. **AI chatbots** explain things well, but they are not reliable with numbers: when researchers gave the best AI models everything needed to fill in simplified US tax returns, the models got fewer than a third of them fully right.

**What we built.** A chat website where a young earner can ask anything about their tax and money, in their own words. They can learn what a term means, get their exact figure, and understand what that figure means for them, all in one conversation. We built it around three made-up people who represent three situations the tax law treats differently: Priya (salaried), Arjun (runs a small business) and Rohan (freelancer).

**The one big idea.** The AI is allowed to understand and to talk, but it is never allowed to calculate. Every number the user sees comes from a calculator program we wrote ourselves, which reads the tax rules from a file. The AI's job is only to decide which calculation to run and then describe the result in plain words. This rule is not just an instruction to the AI; it is enforced by the code in five separate places, so the AI could not break it even if it tried.

**What that gives us.** Every figure can be traced to the exact rule that produced it. The same question gives the same figures every time. And if the AI service is down, the numbers are still correct; the answers just become plainer.

**What it is not.** It is not connected to any bank or to the tax department's website. The three people are invented. The tax rules are for financial year 2025-26 only. It is a working laboratory prototype, not a product anyone could use for their real taxes today.

## 2. How it works

### 2.1 The restaurant analogy

The easiest way to picture the system is a restaurant where the waiters are not allowed to cook.

| In a restaurant                                          | In our platform                              | What it does                                                                                                                         |
|----------------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| The dining room                                          | The website (frontend)                       | What you see and touch: the chat, the list of past chats, the cards                                                                  |
| The front door                                           | /api/chat                                    | The only way an order gets in. Every question, even one started by clicking a card, comes through here                               |
| The manager at the door                                  | The orchestrator                             | Reads your order and sends it to the right waiter                                                                                    |
| Three waiters, each allowed to order only certain dishes | Three agents: Tutor, Computation, Management | They talk to you. Each can request only the calculations on its own list                                                             |
| The order window with a checklist                        | The tool registry ("the doorway")            | Checks the dish exists, that this waiter may order it, and that the order is filled in properly. Then it logs the order              |
| The kitchen                                              | The rule engine                              | Does all the actual cooking, meaning all the arithmetic                                                                              |
| The recipe book                                          | The rulebook, data/tax_rules.json            | Every tax rate, slab, limit and deadline                                                                                             |
| The slip handed back to the waiter                       | "Facts"                                      | A short list of the numbers the waiter is allowed to quote                                                                           |
| An inspector who reads what the waiter is about to say   | The guard                                    | If the waiter mentions any price that is not on the slip, the words are thrown away and a plain, correct description is used instead |
| The storeroom and records                                | The database and data files                  | The people's details, their transactions, past chats, and the explanations                                                           |
| The logbook                                              | The audit log                                | One line for every calculation ever made                                                                                             |

The key rule of this restaurant: **the waiter never cooks**. A waiter can be charming and helpful, but prices come only from the kitchen. That is our platform: the AI is the waiter, the rule engine is the kitchen.

Why separate the waiters by what they are allowed to order, instead of just telling them the rules? Because an instruction can be ignored, but a locked door cannot be walked through. The Tutor agent, which only explains concepts, has no tax calculation on its list at all, so it physically cannot produce a tax figure, whatever it is asked.

### 2.2 The five layers

The system is built in five layers, like the floors of a building. The rule is that each floor may only call the floor directly below it, never the one above. This is what lets us test all the tax arithmetic on its own, with no AI, no internet and no website involved.

## System architecture

a layer calls only the layer below it

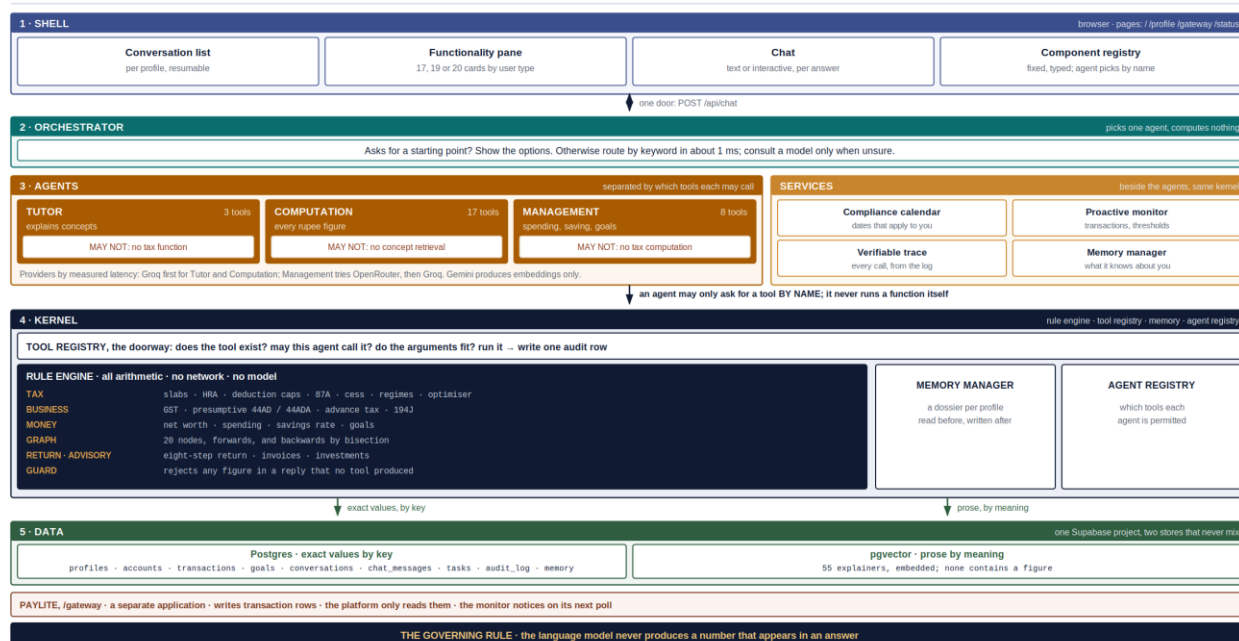


Figure 1. The five layers. Each layer may only call the layer below it.

- Shell (the website).** Runs in your browser. Three panes: your list of chats on the left, a menu of things you can do in the middle (17 cards for Priya, 19 for Arjun, 20 for Rohan), and the chat on the right. Every answer can be shown as plain text or as an interactive card, and has a panel showing exactly how it was worked out.
- Orchestrator (the manager).** Decides which agent should handle the question. It first looks for obvious keywords: a question containing "regime" goes straight to the Computation agent in about a millisecond, with no AI involved. Only if the keywords are unclear does it ask the AI to decide. We do it this way because we measured it: keywords were faster and right, and the AI once sent a clear tax question to the wrong agent.
- Agents (the waiters).** Three of them. The **Tutor** explains concepts and has 3 tools, none of which can calculate tax. The **Computation** agent handles anything whose answer is a number and has 17 tools. The **Management** agent handles spending, saving and goals and has 8 tools. There are 24 tools in total, with a few shared.
- Kernel (the kitchen and its office).** The heart of the system. It holds the rule engine that does all the arithmetic, the tool registry that every request must pass through, the memory manager that remembers things about each person, and the agent registry that lists which tools each agent may use. The word "kernel" is borrowed from operating systems; see the glossary.
- Data (the storeroom).** One online database (Supabase) holding two separate stores: ordinary tables for exact values like balances and transactions, and a special store that finds explanations by meaning. The two never mix, on purpose: you should never get a tax number from a search "by meaning", because that kind of search is approximate.

Beside the agents sit four **services** that watch and report but never act: a compliance calendar (which deadlines apply to you), a proactive monitor (new transactions, unusually large payments, getting close to the GST registration limit), a verifiable trace (a record of every calculation, read from the logbook) and the memory manager. They check every 30 minutes while the profile page is open.

Outside all five layers is **PayLite**, a separate little payments app at /gateway. It can move pretend money between the demonstration bank accounts. The platform can only read those transactions, never make them, which is the same shape as India's Account Aggregator system. It exists so the monitor has something real to notice.

## 2.3 The five safety checks

The rule "no figure shown to the user comes from the AI" is enforced in five places. Each one catches a mistake the others would miss.

1. **The tool forms.** Each calculation accepts only certain inputs. None of them accepts your salary, balance or turnover from the AI; the calculation reads those from your stored profile itself. The most the AI can pass in is a choice, such as which regime, or an amount you yourself mentioned, such as "what if I invest ₹50,000".
2. **The permission check.** Every request goes through one function that refuses any calculation not on that agent's list, even if the AI invents a name for one.
3. **Facts only.** After a calculation runs, the AI is not shown the full result. It gets a short list of named numbers, and nothing else.
4. **The guard.** Before a reply reaches you, every number in it is compared with those facts. If even one number is not there, the whole reply is thrown away and replaced with a plain sentence built from the facts by a fixed template. It also catches numbers written out in words, and the AI doing its own arithmetic on real numbers.
5. **Verdicts written by code.** Some results carry a yes-or-no meaning that the AI could flip while using only correct numbers, such as "this target cannot be reached". For these, the sentence is written by the program, not the AI.

**The one gap, stated honestly.** None of these checks can tell whether the AI picked the right calculation, or passed in the right amount. If you say "invest another ₹50,000" and the AI passes ₹50,000 as the total instead of adding it to what you already have, the calculator correctly answers a slightly different question. The number is right; the question is wrong. Our evaluation measured exactly this (Part 9).

### 3. Frontend, backend and APIs

**Frontend** means everything that runs inside your web browser: the screens, buttons and cards.

**Backend** means everything that runs on the server: the orchestrator, the agents, the calculations and the database access. The frontend never calculates anything. It only sends your question and draws the answer it gets back.

**An API** is a set of addresses the frontend can send requests to, like a counter with a sign saying what you can ask for there. Our project is built with Next.js, which keeps the frontend and backend in one project: files named `page.tsx` are screens, and files inside `app/api/` are backend addresses.

| Where                              | What it is                                                                                                         | Frontend or backend |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------|
| <code>app/page.tsx</code>          | The main chat screen with its three panes                                                                          | Frontend            |
| <code>app/profile/page.tsx</code>  | Money, accounts, goals, deadlines, the watching services, memory                                                   | Frontend            |
| <code>app/gateway/page.tsx</code>  | PayLite, the separate payments app                                                                                 | Frontend            |
| <code>app/status/page.tsx</code>   | A build dashboard showing the layers and tools                                                                     | Frontend            |
| <code>components/</code>           | The building blocks those screens are made of; <code>components/widgets/</code> holds the interactive answer cards | Frontend            |
| <code>app/api/chat</code>          | The main door. Every question goes here: route, run, check, store, return                                          | Backend (API)       |
| <code>app/api/conversations</code> | List, open and delete a person's saved chats                                                                       | Backend (API)       |
| <code>app/api/capabilities</code>  | What this person can ask about, and which cards they have used                                                     | Backend (API)       |
| <code>app/api/memory</code>        | Read or clear what the system remembers about a person                                                             | Backend (API)       |
| <code>app/api/daemons</code>       | One check of the watching services                                                                                 | Backend (API)       |
| <code>app/api/gateway</code>       | PayLite's own door; writes pretend transactions                                                                    | Backend (API)       |
| <code>lib/</code>                  | The orchestrator, agents, kernel and database code                                                                 | Backend             |

Our backend also talks to **outside APIs**, which belong to other companies: the AI providers (Groq, OpenRouter and Google) and Supabase, which hosts our database. Each needs a secret key, kept in a file called `.env.local` on your laptop that never goes to GitHub.

Put together, one request travels like this: **browser** → **/api/chat** → **orchestrator** → **agent** → **(AI service picks the calculation)** → **tool registry** → **rule engine** → **facts** → **(AI service writes a sentence)** → **guard** → **saved to database** → **back to the browser**. The AI service is involved at exactly two points: choosing, and wording.

## 4. One question, followed end to end

Priya types: **"Which regime is better for me?"** Here is everything that happens, with the file where each step lives.

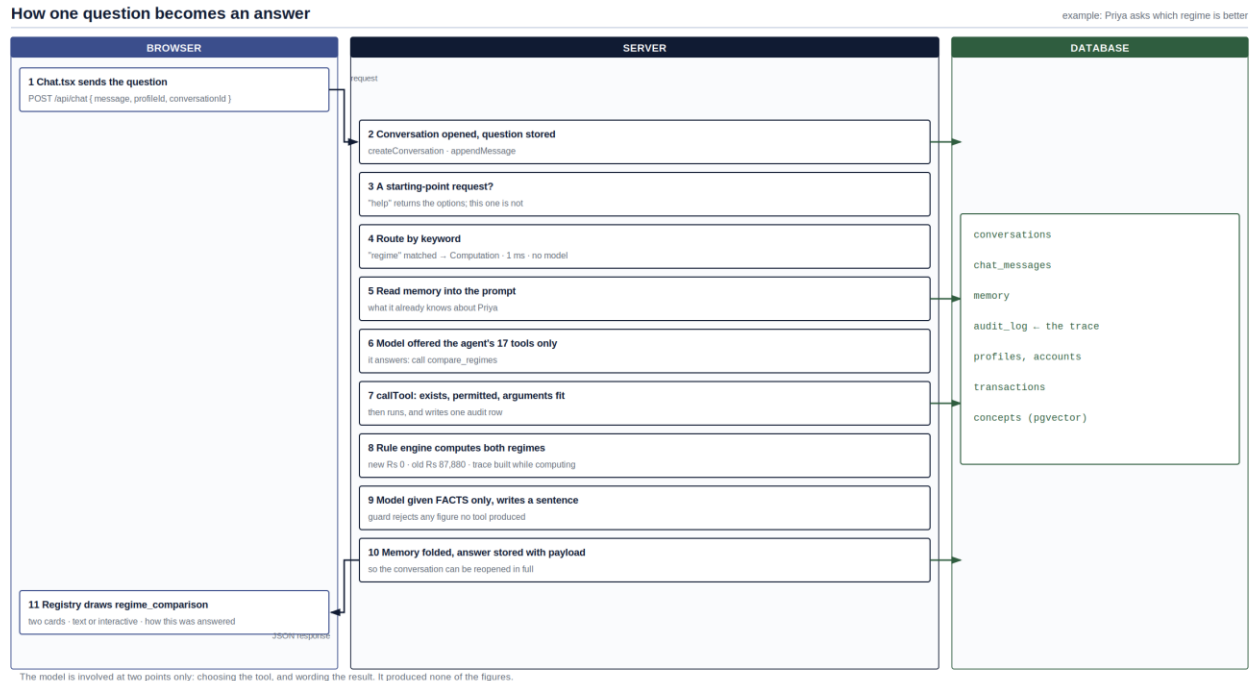


Figure 2. How one question becomes an answer.

1. **The browser sends it.** components/Chat.tsx sends the message, Priya's profile and the current chat to /api/chat.
2. **The question is saved first.** app/api/chat/route.ts opens the conversation (creating and titling it if it is new) and stores the question before anything is calculated.
3. **Is it really a question?** Messages like "help" or "I don't know where to start" skip everything and return suggested starting points. This one is a real question.
4. **Routing.** lib/orchestrator/route.ts sees the word "regime" and sends it to the Computation agent. About one millisecond, no AI.
5. **Memory is read.** lib/kernel/memory.ts condenses what the system already knows about Priya (preferences, what it has explained before) into a few lines.
6. **The AI is offered only this agent's tools.** lib/agents/run.ts shows the AI the Computation agent's 17 tools and nothing else. The AI replies with a request: "run compare\_regimes".
7. **The doorway.** lib/kernel/tools/execute.ts checks the tool exists, that Computation may use it, and that the request is filled in properly. Then it runs it and writes one line to the audit log.
8. **The arithmetic.** The rule engine (lib/kernel/tax.ts and its neighbours) computes Priya's tax under both regimes, writing down its working as it goes.
9. **Facts to the AI.** The AI gets only a short list, such as newRegimeTax 0 and oldRegimeTax 87880. It writes two sentences. lib/kernel/guard.ts puts the numbers into Indian format and checks each one against the facts.
10. **Memory and history are updated.** What ran is recorded against Priya, and the full answer is saved so the chat can be reopened later exactly as it was.
11. **The browser draws it.** The server names a card (regime\_comparison); components/widgets/index.tsx looks that name up in a fixed list and draws it. Priya can switch between text and card, or open "how this was answered".

Priya's actual numbers, so you can see what the kitchen did:

|                             | New regime               | Old regime |
|-----------------------------|--------------------------|------------|
| Gross salary                | ₹12,00,000               | ₹12,00,000 |
| Standard deduction          | ₹75,000                  | ₹50,000    |
| HRA exemption (rent relief) | not allowed              | ₹2,40,000  |
| 80C investments             | not allowed              | ₹50,000    |
| Taxable income              | ₹11,25,000               | ₹8,60,000  |
| Tax before rebate           | ₹52,500                  | ₹84,500    |
| Section 87A rebate          | ₹52,500 (cancels it all) | none       |
| Cess at 4%                  | ₹0                       | ₹3,380     |
| Total tax                   | ₹0                       | ₹87,880    |

The new regime saves her ₹87,880. Notice that her zero depends on three rules working together: the standard deduction, the slab table and the 87A rebate. A mistake in any one of them would change the answer, which is exactly why a tool computes it rather than an AI.

## 5. A tour of the project folder

The repository (the project folder on GitHub and on your Desktop) is organised like this. You do not need to know every file; the ones listed are the ones that matter.

| Folder            | What lives there                                   | Files worth knowing                                                                                                                                                          |
|-------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app/              | Screens and backend doors (APIs)                   | page.tsx (chat), api/chat/route.ts (the main door)                                                                                                                           |
| components/       | The pieces the screens are built from              | Chat.tsx, FunctionPane.tsx, AnswerDetail.tsx; widgets/index.tsx is the component registry                                                                                    |
| lib/orchestrator/ | Layer 2, the manager                               | route.ts (keyword routing), index.ts (route, then run the agent)                                                                                                             |
| lib/agents/       | Layer 3, the waiters                               | providers.ts (which AI service, time limits), prompts.ts (instructions), run.ts (runs an agent, applies the guard, falls back), fallback.ts and describe.ts (the no-AI path) |
| lib/kernel/       | Layer 4, the kitchen                               | tax.ts, slabs.ts, hra.ts, regimes.ts, business.ts (GST, 44AD, 44ADA, advance tax, 194J), guard.ts, agents.ts (who may use what), memory.ts                                   |
| lib/kernel/tools/ | All the tools and the doorway                      | registry.ts (all 24 tools), execute.ts (the doorway)                                                                                                                         |
| lib/db/           | Layer 5, talking to the database                   | schema.ts (the 9 tables), repositories.ts (every read and write, with the backup to seed files)                                                                              |
| data/             | The fixed data                                     | tax_rules.json (the rulebook), profiles.json (the three people), concepts.json (55 explanations), investments.json (8 options)                                               |
| scripts/          | Command-line tools                                 | verify.ts, ask.ts, checklist.ts and others, run with npm run                                                                                                                 |
| tests/            | The 318 automated checks, in 18 files              | corrections.test.ts holds the two tax fixes                                                                                                                                  |
| eval/             | The evaluation for the paper                       | questions.json (30 questions and answers), run-eval.ts, score.ts, results/                                                                                                   |
| docs/             | The master plan in text form, and the two diagrams | MASTER_PLAN.md                                                                                                                                                               |
| paper/            | Everything for writing the paper                   | PAPER_STATE.md, the chat prompt, source table, literature notes, abstract                                                                                                    |
| Root              | Settings and overview files                        | README.md, HOW_IT_WORKS.md, package.json (list of commands), .env.example                                                                                                    |

Two things are deliberately **not** in the repository: **.env.local**, which holds your secret keys and database password, and **node\_modules**, the downloaded libraries, which npm install recreates on any machine.

## 6. Our data

### 6.1 What data we have

| Data                                        | What it is                                                                                                                                          | Real or made up                 |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| The rulebook, data/tax_rules.json           | Every rate, slab, limit, threshold and deadline for financial year 2025-26, typed in by hand. The only place tax numbers come from                  | Real law (Income-tax Act, 1961) |
| Three people, data/profiles.json            | Priya, Arjun and Rohan: income, rent, city, deductions already claimed                                                                              | Made up, but realistic          |
| Bank accounts, transactions, goals          | Sample money history for each person, loaded into the database. PayLite can add more                                                                | Made up                         |
| 55 explanations, data/concepts.json         | Short plain-language explanations of tax and money ideas. None contains a single number, on purpose, because the Tutor may quote them word for word | Written for the project         |
| 8 investment options, data/investments.json | PPF, ELSS, NPS and so on, with lock-in and tax treatment. No return rates, because that would be a number with no rule behind it                    | Real product features           |
| Chats, messages, memory, audit log          | Created as the platform is used                                                                                                                     | Generated while running         |

### 6.2 The three people

|                      | Priya, 24                                          | Arjun, 28                                                         | Rohan, 26                                                              |
|----------------------|----------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| Situation            | Salaried software engineer, Bengaluru              | Runs a digital services business, Mumbai                          | Freelance designer, Pune                                               |
| Income               | Salary ₹12,00,000                                  | Turnover ₹24,00,000, expenses ₹9,00,000                           | Receipts ₹18,00,000 (₹10,00,000 from overseas), expenses ₹4,00,000     |
| What she or he tests | Regime choice, HRA, 87A rebate, preparing a return | GST registration and payment, presumptive tax (44AD), advance tax | Presumptive tax (44ADA), GST headroom, TDS by clients (194J), invoices |
| A key figure         | Old regime tax ₹87,880, new regime ₹0              | 44AD profit ₹1,46,400; GST payable ₹3,40,000                      | 44ADA profit ₹9,00,000; ₹2,00,000 below the GST limit                  |

Each person sits deliberately on a boundary, because a test case comfortably inside every range tests nothing. Where the source material only gave a gross salary, we made stated assumptions: basic salary is half the gross, HRA is half the basic, Bengaluru and Pune count as non-metro. These are recorded in the profiles file.

### 6.3 Where the data is stored

Everything lives in one Supabase project, which is an online Postgres database. It holds nine ordinary tables (profiles, accounts, transactions, goals, conversations, chat messages, tasks, audit log, memory) and one special table for the 55 explanations, searched by meaning using pgvector.

**If the database is unreachable**, every read falls back to the seed files in data/. The tax figures stay correct because they only need the rulebook and the profiles. What you lose is chat history and transactions.

### 6.4 What it can realistically do, and what it cannot

**It can:** compare the two regimes; compute tax with the full working; work out HRA relief; find unused deductions; answer what-if questions; work backwards from a tax you want to the investment needed; estimate GST and whether registration is required; compare presumptive tax with regular books; plan advance tax; check what clients should have deducted; draft an invoice; prepare a return from Form 16 figures (as a file, not submitted anywhere); report net worth, spending, savings rate and goals; list the deadlines that apply to you; and explain any term.

**It cannot:** read a real bank account; file anything with the government; handle any year other than 2025-26 or the new Income Tax Act, 2025; handle surcharge (incomes above ₹50 lakh), capital gains or senior citizen rules; apply the new regime's 87A marginal relief; tell whether a deduction actually applies to you (it assumes you could use any of them); log users in; or work in any language except English.

## 7. The technology, one line each

| Technology                       | What it does for us                                                                            | Why this one                                                                                          |
|----------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| TypeScript                       | The programming language for the whole project                                                 | Catches mistakes before the program runs, which matters most when a wrong number is the worst outcome |
| Node.js and npm                  | Runs JavaScript and TypeScript outside a browser; npm installs libraries and runs our commands | The standard toolset for this kind of project                                                         |
| Next.js                          | Web framework holding both the screens and the backend                                         | One project, one command to start                                                                     |
| React                            | Builds the screens out of reusable pieces                                                      | The component registry depends on it                                                                  |
| Tailwind CSS                     | Styling: colours, spacing, layout                                                              | Styles written right next to the screen code                                                          |
| Vercel AI SDK                    | One way of talking to several AI providers, with tool calling built in                         | Swapping providers needs no code change                                                               |
| Groq, model openai/gpt-oss-20b   | The main AI for routing when unsure, and for choosing and wording                              | Measured fastest with working tool calls, on a free tier                                              |
| OpenRouter                       | A backup route to many AI models                                                               | Configured fallback; its free models change often                                                     |
| Google Gemini                    | Turns text into embeddings for search by meaning                                               | Used only where its slower speed does not keep anyone waiting                                         |
| Supabase (Postgres and pgvector) | The online database, including search by meaning                                               | Both kinds of storage in one free project                                                             |
| Drizzle                          | Describes the database tables in TypeScript                                                    | The table design lives with the code                                                                  |
| Zod                              | Declares exactly what each tool accepts                                                        | This is what enforces the first safety check                                                          |
| Vitest                           | Runs the 318 automated checks                                                                  | Fast, and needs no AI or internet                                                                     |
| tsx                              | Runs TypeScript scripts from the terminal                                                      | Lets npm run eval and friends work directly                                                           |
| Git and GitHub                   | Saves every version of the project and shares it                                               | How your laptop and this chat stay in sync                                                            |

Everything runs on free tiers. The AI model names are read from settings rather than written into the code, because providers retire models without warning; this happened during development.

## 8. Everything we ran these last few days, and why

The pattern each time was the same. I wrote a script and tested it on a fresh copy of your repository. You downloaded it and ran it from the project folder, which changed some files. Then git add, commit and push saved those changes to GitHub, where I could read them.

| What ran                                                        | What it did                                                                                                                                                                                                                                                                                               | Why                                                                                                                                                      |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| fix-audit.sh (before this chat)                                 | Fixed the invoice form, stopped the AI from filling in Form 16 figures, corrected labels, removed an unused file                                                                                                                                                                                          | A pre-paper check of the code found them. Tests went to 314                                                                                              |
| fix-rules.sh                                                    | Two tax corrections. Section 44AD: 6% applies only to digital receipts and 8% to the rest, so Arjun's figure is ₹1,46,400, not ₹1,44,000. Section 194J: only Indian clients deduct TDS, so Rohan's clients should have deducted ₹80,000, not ₹1,80,000, and nothing is missing. Added 4 tests, making 318 | Found while I worked out the evaluation's answers by hand: two answers did not match what the platform said, and the platform was the one that was wrong |
| eval-setup.sh                                                   | Added the eval/ folder: the 30 questions with their correct answers, the program that asks them, the marking rules, the summary tables and the guard test. Added three commands: npm run eval, eval:guard and eval:summary                                                                                | The paper needs measured evidence                                                                                                                        |
| Checking the answer key                                         | You checked all 30 correct answers against their working. Your name is recorded on each                                                                                                                                                                                                                   | The paper says the answer key was checked by an author, so someone had to                                                                                |
| npm run eval:guard                                              | The guard stress test: about 1,700 made-up replies, no AI needed, a few seconds                                                                                                                                                                                                                           | Tests the guard directly                                                                                                                                 |
| npm run eval, first time                                        | Asked every question in all four ways. Problem: the platform gives the AI 12 seconds before giving up, the AI was slow that day, and 77 of 90 platform answers timed out and fell back to the no-AI path                                                                                                  | Not a fair test: the plain-AI comparison had been given 60 seconds                                                                                       |
| MODEL_TIMEOUT_MS=60000<br>npm run eval -- --only platform-model | Re-ran only the platform-with-AI answers, with the same 60 seconds                                                                                                                                                                                                                                        | Fairness. Partway through, Groq's free daily limit ran out                                                                                               |
| git checkout c5c46c2 -- ...                                     | Restored the 75 answers already saved, after the pasted block's first line had deleted them; then the run finished the last 15                                                                                                                                                                            | Recovering from the accidental second paste. Nothing was lost                                                                                            |
| npm run eval:summary                                            | Marks every stored answer and writes summary.md and answers.csv                                                                                                                                                                                                                                           | Turns raw answers into the tables. While reading them I found and fixed two faults in my marking rules, both about reading a zero                        |
| paper-setup.sh                                                  | Made the paper/ folder, moved the documents in, updated the master plan with the results, applied the marking fix and re-marked                                                                                                                                                                           | Tidy repository, documents that match reality                                                                                                            |
| final-docs.sh                                                   | Put the last three reference DOIs in the source table and froze the paper state                                                                                                                                                                                                                           | Documents complete                                                                                                                                       |

### 8.1 The two sets of numbers people mix up

**318 automated tests** (run with npm test) are small programs that check the code does what it should. Examples: "Priya's old-regime tax must come out as ₹87,880", "the Tutor must be refused if it asks for a

tax tool", "the guard must reject a reply containing a made-up number". They take seconds, involve no AI and no internet, and all 318 must pass. They check the **code**.

**360 evaluation answers** are 30 questions, each asked in 4 different ways, 3 times each. They check how the **whole system, AI included**, behaves, and they produce the numbers in the paper. When we say "90", we mean the 90 answers for one of the four ways (30 questions times 3 runs).

## 9. The evaluation, and how the numbers were made

### 9.1 Why we did it

Before this, the project had a design and a working system but almost no measured evidence: a handful of one-off observations. A paper has to show, with numbers, what the design does and does not achieve. So we built a small, fair, repeatable test.

### 9.2 The questions and the answer key

30 realistic questions, 10 for each person, each with exactly one correct answer: a rupee figure, zero, yes or no, or "cannot be reached". Example, question P08, asked as Priya: **"If my total 80C investment becomes Rs 1,50,000, what will my old-regime tax be?"** Correct answer: **₹67,080**.

The correct answers (the **answer key**) were worked out by hand from the rules, not by running the platform. That matters: if we had asked the platform for the correct answers, we would have been checking it against itself, and its mistakes would have looked right. Working them independently is exactly how the 44AD and 194J errors were caught.

### 9.3 The four ways of asking

| Way ("condition")               | Who answers                                                                                                   | What it tests                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Platform with AI                | The platform as built, 60 seconds allowed per AI call                                                         | The real system                                                                |
| Platform without AI             | The same platform with every AI key removed                                                                   | The claim that figures stay correct when the AI is unavailable                 |
| AI with rulebook (baseline)     | The same AI, no tools, given the person's profile and the whole rulebook, told to finish with "ANSWER: value" | Whether the AI alone can get it right when it has all the rules in front of it |
| AI with profile only (baseline) | As above, without the rulebook                                                                                | What a plain chatbot does                                                      |

A **baseline** is the thing you compare against, so your numbers mean something. "The platform got 69 of 90" means little alone. "The platform got 69, and the same AI working alone got 53 even with every rule in front of it" is a finding. We used the **same AI** in every condition so that any difference comes from the design, not from a better or worse model. We gave one baseline the whole rulebook so nobody can say the AI simply did not know the 2025-26 rules.

Each question was asked **three times** in each way, because AI answers vary from one attempt to the next. Three runs show whether the answers are consistent.

### 9.4 How an answer was marked

Marking was done by a program (eval/score.ts), the same way for every answer, so nobody's judgement could tilt it.

- **Platform:** right only if the calculation that answers the question produced the correct value **and** the reply shown to the user actually states it. Quoting a correct number for the wrong reason does not count.
- **Baseline:** right only if its final "ANSWER:" line is the correct value. A correct number mentioned in passing earlier does not count, and a reply with no final line counts as wrong.

### 9.5 What each column of the results means

- **Right, of 90:** of the 90 answers in that way of asking, how many were correct.

- **Questions right in all 3 runs:** of the 30 questions, how many were answered correctly every single time.
- **Same answer in all 3 runs:** of the 30 questions, how many got the same answer all three times, right or wrong. This measures consistency.
- **Replies with a figure no tool produced:** of the platform's replies, how many contained any number (above 100) that the calculator did not produce. This is the safety measure, and the heart of the paper.

## 9.6 The results

|                      | Right, of 90 | Right in all 3 runs | Same answer in all 3 runs | Figures no tool produced |
|----------------------|--------------|---------------------|---------------------------|--------------------------|
| Platform with AI     | 69           | 20 of 30            | 24 of 30                  | 0 of 90                  |
| Platform without AI  | 54           | 18 of 30            | 30 of 30                  | 0 of 90                  |
| AI with rulebook     | 53           | 15 of 30            | 19 of 30                  | not applicable           |
| AI with profile only | 23           | 4 of 30             | 6 of 30                   | not applicable           |

### What this honestly shows:

- Across 180 replies, with and without the AI, the platform never showed a number its calculator did not produce.
- The platform with AI was right 69 times out of 90, against 53 for the same AI working alone with every rule, and 23 with only the profile.
- The advantage comes from Arjun and Rohan, the business owner and the freelancer (23 against 12, and 21 against 15). For Priya, the salaried case, the AI with the rulebook did as well as the platform (26 against 25). The paper must say this.
- **How the platform was wrong (21 times):** 6 times the AI misread "invest another ₹50,000" as the total, so the calculator correctly answered a slightly different question. 15 times the AI picked the wrong calculation, or none, or asked for information the platform already had. Once, it wrote a false sentence with no number in it ("the new regime applies only to salaried people"), which the guard cannot catch because it only checks numbers.
- **How the baseline was wrong:** every wrong answer that reached a final line was a wrong number stated as the answer. With the rulebook, 22 of its 75 finished answers were wrong numbers. Another 15 never finished at all.
- **Without AI:** 54 of 90, identical every time, every number correct. It gets the other questions wrong because it answers a related question instead, usually the regime comparison.

## 9.7 The guard stress test

Separately, a program built 1,707 fake replies around real calculator outputs: some honest (correct numbers written in different styles, like 24,00,000, 2,400,000, 24 lakh, 2.4 million), and some with a planted mistake (one digit changed, one rupee off, rounded, two real numbers added, a number from another person, a number written in words). Then it ran each through the guard, exactly as a real reply would be checked.

Result: **0 of 448 honest replies rejected; 1,062 of 1,062 planted mistakes above ₹100 caught.** The known blind spot: numbers of 100 or less are let through by design, so ordinary words like "section 80" or "12 percent" do not trip it. A made-up small amount would slip through. These replies were constructed by a program, not written by the AI, and the paper will say so.

## 9.8 The limits of all this

One AI model, one day of runs, 30 questions, three invented people, and an answer key checked by one person. That supports statements like "on these 30 questions, the platform got 69 of 90". It does not support "the platform is 77% accurate" in general, and we will not claim that.

## 10. The paper

### 10.1 Where it is going

**BAICONF 2026**, the 13th Business Analytics and Intelligence Conference at IIM Bangalore, 17 to 19 December 2026. Our abstract was accepted (submitted under Dr. Kalaavathi's name). The full paper is due **31 October 2026**: at most 4,500 words, with an introduction, literature review, methodology, findings, conclusion and implications, and references. They would like the data set as an Excel file. One of us must register by **14 November** (₹5,000 at the student rate). We dropped the earlier plan to send it to ICADCML (Springer), because the same paper cannot go to two conferences.

### 10.2 What kind of paper this is

An **applied systems paper**: we describe a design, explain why it is built that way, and test it with a small, controlled evaluation. It is not a theory paper, not a study of real users, and not a large benchmark. The audience is analytics and management people, so it also needs a proper section on what this means in practice for fintech builders, tax educators and regulators.

### 10.3 The aim

To show that a financial assistant can use AI for language while guaranteeing that every number shown came from verified rules, and to measure honestly what that design buys and what it does not.

### 10.4 Is there novelty?

Yes, but it has to be stated precisely, because several nearby things already exist.

#### Not new:

- Letting an AI hand arithmetic to a calculator or to code it writes (published work called PAL, Program of Thoughts, Toolformer).
- Systems with several AI agents, and operating-system style designs for them (for example AIOS).
- Combining Indian tax rules with a chatbot (a 2026 system called Auto ITR, and the tax department's own assistant, Kar Saathi).

#### New in ours:

- The rule is **enforced and checked on every reply** at five points, not just suggested to the AI.
- The AI **never supplies the quantities**. In the program-writing approaches, the AI copies numbers into its program, and one of those studies found that most of its remaining errors on financial questions were exactly that copying. Our calculations read personal figures from stored data instead.
- Agents are separated by **what they are able to do**, not by what they are told.
- It is **measured against the same AI without the design**, on Indian tax, where we found no published benchmark at all.
- It reports the failures that remain, in particular that the AI can still pick the wrong calculation or misread an amount.

### 10.5 What we may and may not claim

**May claim:** the design and its five checks; 0 of 180 platform replies with a number the calculator did not produce; 69 of 90 against 53 and 23, always with the sample stated; the platform's mistakes being unanswered questions rather than wrong numbers; the consistency figures; correct figures without the AI; the guard stress test as constructed replies; the two rule errors caught by independent checking.

**May not claim:** that it is the first of its kind; any general accuracy percentage; that it beats the plain AI on every question (it did not, for Priya); that it improves anyone's financial literacy (we never tested that with people); that AI models make serious errors on Indian tax specifically (all the published evidence is on US and other countries' law); that the guard catches every false statement; anything about speed; anything about production use, security, other years or the new Act.

## 10.6 How it gets written

1. Show everything to Dr. Kalaavathi, and get her approval to correct the one abstract sentence that says tax knowledge is the weakest area, which the survey report does not support.
2. Open a new chat inside the Capstone project and paste the prompt from paper/PAPER\_CHAT\_PROMPT.md. It reads all the documents, then proposes a plan.
3. After you approve the plan, it writes the full paper as a Word file within the 4,500 words, plus the Excel data set and the two figures. Every section gets a pass to make it read naturally.
4. You, your teammates and the guide read it; comments are folded in.
5. You submit it by 31 October and register by 14 November.

Everything the writing needs is already in the paper/ folder and the project files: the master plan (section 15.3 has the full evaluation, and section 17 the claims rules), the paper state, the literature notes with the exact sentences for each related paper, and the checked source table.

## 11. Glossary

Every term is explained in plain words first. Where it helps, **In our project** says exactly where it shows up, and **Not the same as** heads off a common mix-up.

### 11.1 AI words

#### Artificial intelligence (AI)

Software that does things we normally think need a person, like understanding a sentence or writing a reply. In this document, "the AI" always means the language model described next.

*In our project:* The AI is one component among many. Most of the platform is ordinary code that follows exact rules.

#### Large language model (LLM), "the model"

An AI trained on huge amounts of text to predict what words should come next. That makes it very good at understanding questions and writing fluent answers. It does not calculate: when it writes "₹87,880", it is producing text that looks like a calculation result, which may or may not be correct.

*In our project:* We use a model called openai/gpt-oss-20b, run by a company called Groq. It is used only to choose a calculation and to put the result into words.

*Not the same as:* The model is not the agent. An agent is the model plus instructions plus a list of allowed tools.

#### Prompt

The text given to a model: instructions plus the question. The model's reply depends on it.

*In our project:* Each agent's instructions are in lib/agents/prompts.ts. The baseline's prompt in the evaluation contained the profile, the question, and (in one condition) the whole rulebook.

#### Hallucination

When a model states something false as if it were true, such as an invented number or a made-up rule. It is not lying; it is producing plausible text with nothing checking it against reality.

*In our project:* The guard exists to stop hallucinated numbers reaching the user. It cannot stop a hallucinated sentence that contains no number, which happened once in the evaluation.

#### Agent

A model given a job, instructions, and a list of tools it is allowed to request. It reads the question, decides which tool to ask for, and writes the answer from the tool's result.

*In our project:* Three agents: Tutor (explains, 3 tools), Computation (numbers, 17 tools), Management (money habits, 8 tools).

*Not the same as:* An agent is not a separate AI. All three use the same model with different instructions and different tool lists.

#### Multi-agent system

A system where several agents each handle a different kind of job, with something deciding which one handles each request.

*In our project:* Our three agents plus the orchestrator that picks between them.

#### Orchestrator

The part that looks at each incoming question and decides which agent should handle it. It never answers anything itself.

*In our project:* lib/orchestrator/. It checks keywords first and asks the model only when unsure.

#### Routing

The act of sending a question to the right place.

*In our project:* "Keyword routing" means deciding from words in the question, such as "regime" or "spend". It takes about a millisecond and involves no AI.

## Tool

A specific function the model is allowed to ask for by name, like "compare\_regimes" or "compute\_gst". The model sees the tool's name, a description and what inputs it takes, but never its code.

*In our project:* 24 tools, all listed in lib/kernel/tools/registry.ts.

*Not the same as:* A tool is not an app like Excel. It is one named calculation or lookup inside our code.

## Tool call (function calling)

The model's way of saying "please run this tool with these inputs". Our code, not the model, then runs it.

*In our project:* Every tool call passes through the doorway (lib/kernel/tools/execute.ts) and leaves one line in the audit log.

## Arguments

The inputs passed to a tool, like which regime, or an amount.

*In our project:* The model may only pass choices or amounts the user stated. Personal figures like salary are read by the tool itself from stored data. A wrong argument ("another ₹50,000" passed as the total) was the cause of 6 of the platform's wrong answers.

## Embedding

A list of numbers that represents the meaning of a piece of text, so that texts with similar meanings get similar lists. Used to search by meaning rather than by exact words.

*In our project:* The 55 explanations are turned into embeddings by Google Gemini so the Tutor can find the right one even if your wording differs.

## Search by meaning (retrieval, RAG)

Finding text by what it means instead of exact word matches. RAG, retrieval-augmented generation, means fetching relevant text first and having the model answer from it.

*In our project:* Only the Tutor uses this, and only for explanations, which contain no numbers.

*Not the same as:* Search by meaning is approximate, which is exactly why tax numbers are never looked up this way.

## AI provider

A company that runs AI models on its computers and lets programs use them over the internet.

*In our project:* Groq (main), OpenRouter (backup), Google Gemini (embeddings only).

## API key

A secret password that lets our program use a provider's service. Anyone with it can use your account.

*In our project:* Kept in .env.local on your laptop, never on GitHub.

*Not the same as:* An API key is not an API. The API is the service; the key is your permission to use it.

## Timeout

A time limit after which the program stops waiting for a reply.

*In our project:* The platform gives the model 12 seconds, then answers without it. In the evaluation we allowed 60 seconds, the same as the baseline, because the model was slow that day.

## Rate limit

A cap a provider puts on how much you can use it per minute or per day, especially on free plans. Hitting it means waiting.

*In our project:* Groq's free daily limit was hit during the evaluation rerun, which is why the run paused and we finished it later.

## Fallback

What the system does when its first choice fails.

*In our project:* When the model is slow, missing or wrong, the platform picks a tool by keyword (lib/agents/fallback.ts) and writes the answer from a fixed template (lib/agents/describe.ts). The numbers stay correct; the wording is plainer and it answers fewer questions directly.

## 11.2 Software words

### Frontend

The part of a website that runs in your browser: what you see and click.

*In our project:* app//page.tsx files and the components/ folder.

### Backend

The part that runs on the server: the logic, the calculations, the database access.

*In our project:* lib/ and the app/api/ folder.

### Server

A computer program that waits for requests and answers them. When you run npm run dev, your laptop acts as the server.

### Browser

Chrome, Safari and the like. The frontend runs here.

### API

A set of addresses a program can send requests to, each doing one job, like counters at a bank.

*In our project:* Our own API is in app/api/ (chat, conversations, capabilities, memory, daemons, gateway). We also use other companies' APIs: the AI providers and Supabase.

### Endpoint

One address in an API, such as /api/chat.

### POST request

The kind of request used to send data to a server, like submitting a form.

*In our project:* The chat screen sends each question to /api/chat as a POST request.

### JSON

A plain-text format for structured data: names and values in curly brackets. Easy for both people and programs to read.

*In our project:* The rulebook, the profiles, the explanations and the questions are all JSON files.

### JSONL

"JSON Lines": one JSON record per line, so records can be added one at a time.

*In our project:* The evaluation saved each answer as one line in eval/results/\*.jsonl, which is why a stopped run could continue where it left off.

### CSV

A plain table where columns are separated by commas. Opens in Excel.

*In our project:* eval/results/answers.csv has every answer, one per row.

### Database

An organised store of data that programs can search and update reliably.

### Table, row, column

How a database is laid out, like a spreadsheet: a table per kind of thing, a row per item, a column per detail.

*In our project:* 9 tables, for example transactions, where each row is one payment.

### Postgres

A widely used free database system.

### Supabase

A company that hosts Postgres databases online, free for small projects.

*In our project:* Our database lives there.

### **pgvector**

An add-on to Postgres that stores embeddings and finds the closest ones.

*In our project:* Holds the 55 explanations for search by meaning.

### **Seed data, seed files**

Starting data loaded into a system so it has something to work with.

*In our project:* The files in data/. If the database is unreachable, the platform reads these directly, which is also how the evaluation ran.

### **Repository (repo)**

A project folder whose full history is tracked by Git.

*In our project:* github.com/Nagasai-Datta/taxwise, and the copy at ~/Desktop/taxwise.

### **Git**

A tool that records every saved version of a project, so you can see changes and go back.

*Not the same as:* Git is not GitHub. Git runs on your laptop; GitHub is a website that stores a copy online.

### **GitHub**

A website that hosts Git repositories online.

*In our project:* How I read your code and results: you push, I pull.

### **git add, commit, push**

Three steps to save work: add chooses which changes to include, commit saves them as a named version on your laptop, and push uploads that version to GitHub.

### **git pull and clone**

Pull downloads new versions from GitHub into an existing copy. Clone makes a fresh full copy.

### **Commit hash**

The short code naming one saved version, like c5c46c2.

*In our project:* We used "git checkout c5c46c2 -- file" to bring back one file exactly as it was in that version.

### **Terminal**

The text window where you type commands instead of clicking.

### **cd**

"Change directory": moves the terminal into a folder.

*In our project:* cd ~/Desktop/taxwise, before every command.

### **Script (.sh file)**

A text file containing a list of terminal commands, run in one go with bash.

*In our project:* Every file I sent you ending in .sh: fix-rules.sh, eval-setup.sh, paper-setup.sh, final-docs.sh.

### **bash**

The program that runs terminal commands and scripts.

*In our project:* bash ~/Downloads/script.sh runs a downloaded script.

### **Node.js**

Lets JavaScript and TypeScript run outside a browser.

### **npm**

Node's package manager: installs libraries (npm install) and runs named commands listed in package.json (npm run something).

*In our project:* npm test, npm run eval, npm run eval:guard, npm run eval:summary, npm run dev.

## Package, library

Ready-made code written by others that a project uses.

*In our project:* Listed in package.json; downloaded into node\_modules/.

## TypeScript

JavaScript with types: you declare what kind of data each thing holds, so many mistakes are caught before running.

*Not the same as:* You write plain JavaScript; TypeScript reads almost the same, with extra type labels.

## Next.js

A framework for building websites where screens and backend live in one project.

## React

A library for building screens out of reusable pieces called components.

## Component

One reusable piece of a screen, like a card or a button.

*In our project:* Each interactive answer card is a component in components/widgets/.

## Component registry

A fixed list mapping names to components.

*In our project:* components/widgets/index.tsx. The server names a card; the browser looks it up. The model never writes screen code, so every card looks the same every time and can be tested.

## Environment variables, .env.local

Settings given to a program from outside its code, often secrets.

*In our project:* .env.local holds the API keys, database address and settings like MODEL\_TIMEOUT\_MS. Setting MODEL\_TIMEOUT\_MS=60000 before a command changes the time limit for that one run.

## Deterministic

Always giving the same output for the same input, with no randomness.

*In our project:* The rule engine is deterministic. The model is not, which is why the evaluation ran each question three times.

## Automated test, assertion

A small program that checks one thing the code must do. An assertion is a single check inside it, such as "this must equal ₹87,880".

*In our project:* 318 assertions in 18 files under tests/, run with npm test. They check the code, not the AI.

## Vitest

The tool that runs our automated tests.

## Refactoring

Reorganising code to make it cleaner without changing what it does.

## caffeinate

A Mac command that stops the laptop sleeping while another command runs.

*In our project:* caffeinate -i npm run eval kept the long evaluation from stopping.

## 11.3 Our system's own words

### The platform

What the paper calls our system. Its registered title is A Conversational Platform for Financial Literacy and Management using a Multi-Agent LLM Orchestrator.

*Not the same as:* "taxwise" is only the folder and repository name; it is not used in the paper.

## The governing rule

No figure shown to the user originates in the language model. Everything else in the design exists to enforce it.

## Shell

Layer 1: the website the user sees.

## Kernel

Layer 4: the core that holds the rule engine, the tool registry, the memory manager and the agent registry.

*Not the same as:* Not the kernel of Windows or macOS. We borrowed the word because, like an operating system's kernel, ours controls what the parts above it are allowed to do.

## Rule engine

All the tax and money arithmetic, written as ordinary code. No AI, no internet.

*In our project:* lib/kernel/: tax.ts, slabs.ts, hra.ts, business.ts and others.

## Rulebook

The file holding every tax rate, slab, limit and deadline.

*In our project:* data/tax\_rules.json, financial year 2025-26. Changing a rate means changing this file, not the code.

*Not the same as:* Not the same as the explainer corpus, which holds explanations and no numbers.

## Versioned rulebook

The rules are stored as data tied to a specific year, so a new year or the new Income Tax Act means writing a new rulebook rather than rewriting code.

## Tool registry, "the doorway"

The single function every tool call must pass through. It checks the tool exists, that this agent may use it, and that the inputs are the right shape; then runs it and logs it.

*In our project:* lib/kernel/tools/execute.ts, using the list in registry.ts.

## Agent registry

The list of which tools each agent may use.

*In our project:* lib/kernel/agents.ts.

## Permission check

The refusal of any tool not on the agent's list. Safety check number 2.

## Facts

The short list of named numbers a tool hands back to the model, like oldRegimeTax 87880. The model may repeat these and nothing else.

## The guard

The final check on every reply: any number not found in the facts gets the reply thrown away and replaced by a template sentence built from the facts.

*In our project:* lib/kernel/guard.ts. Blind spots by design: numbers of 100 or less, and false statements with no numbers.

## Normalising

Rewriting numbers into one standard form before checking them, so "2.4 million", "24 lakh" and "2,400,000" are all recognised as ₹24,00,000.

*In our project:* Stops the guard rejecting a reply that is only written differently.

## Template, deterministic wording

A fixed sentence with blanks filled in from the facts. Used when the model is unavailable or its reply was rejected.

*In our project:* lib/agents/describe.ts.

### Authoritative result

A result whose sentence is written by code, not the model, because the model could flip its meaning. Safety check number 5.

*In our project:* Used for "this target cannot be reached".

### Audit log, trace

One database row per tool call: which tool, which agent, when, how long. The trace shown to users is read from it.

*In our project:* Because every calculation must pass through the doorway, nothing can be computed without leaving a row.

### Working (rule-by-rule trace)

The step-by-step calculation the rule engine records while it computes, shown in "how this was answered". Written during the calculation, not reconstructed afterwards.

### Memory, dossier

What the system remembers about each person: preferences, decisions, concepts already explained.

*In our project:* lib/kernel/memory.ts. Recorded from what actually ran, not from what the model claims.

### Services (daemons)

Background helpers that watch and report but never act: compliance calendar, proactive monitor, verifiable trace, memory manager.

*In our project:* They check every 30 minutes from the browser while the profile page is open.

### PayLite

The separate pretend payments app at /gateway. It moves money between demonstration accounts; the platform only reads the results.

### Explainer corpus

The 55 short explanations the Tutor draws on. None contains a number.

*In our project:* data/concepts.json.

### Dual mode

Every answer can be shown as a sentence or as an interactive card, both drawn from the same result.

### Working backwards, bisection

Finding the input that produces a figure you want, by repeatedly halving the range it must be in.

*In our project:* "How much must I invest for my tax to be ₹40,000?" For Priya the honest answer is that it cannot be reached through 80C.

### Profile, fixture

One of the three made-up people. "Fixture" is the testing word for fixed example data.

*In our project:* Priya (PRIYA-001), Arjun (ARJUN-002), Rohan (ROHAN-003).

### Functionality pane

The middle column listing what this person can do. Clicking a card types a question into the chat; it never calculates by itself.

## 11.4 Evaluation words

### Evaluation

A planned test that produces numbers you can report.

*In our project:* 30 questions, 4 conditions, 3 runs: 360 answers, plus the guard stress test.

*Not the same as:* Not the same as the automated tests. Tests check the code; the evaluation checks the whole system's behaviour, AI included.

### **Answer key, expected answer**

The correct answer to each question, decided before running.

*In our project:* Worked by hand from the rules, not from the platform, and checked by Nagasai.  
Stored in eval/questions.json.

### **Condition**

One way of answering the questions.

*In our project:* Platform with AI, platform without AI, AI with rulebook, AI with profile only.

### **Baseline**

What you compare your system against, so your numbers mean something.

*In our project:* The same AI with no tools, with and without the rulebook.

### **Run**

One pass through all 30 questions in one condition.

*In our project:* Three runs per condition, so 90 answers per condition.

### **Marking, scoring**

Deciding whether each answer is right, by fixed rules applied by a program.

*In our project:* eval/score.ts.

### **ANSWER line**

The final line the baseline was told to write, like "ANSWER: 67080". Only this line was marked, so a correct number mentioned in passing does not count.

### **Right, of 90**

How many of the 90 answers in one condition were correct.

*Not the same as:* This is a count on our 30 questions, not a general accuracy percentage.

### **Right in all 3 runs**

How many of the 30 questions were answered correctly every time.

### **Same answer in all 3 runs (reproducibility)**

How many questions got the identical answer all three times, right or wrong.

### **Figure no tool produced (stray figure)**

Any number above 100 in a platform reply that the calculator did not produce. The safety measure.

*In our project:* Zero in all 180 platform replies.

### **Guard stress test**

The separate test that builds 1,707 fake replies around real outputs and checks what the guard catches.

### **Faithful reply, invented reply**

In the stress test: a faithful reply uses only real figures (in any style); an invented one contains a planted wrong number.

### **Blind spot**

A known gap in what a check can see.

*In our project:* The guard ignores numbers of 100 or less, and cannot judge sentences with no numbers.

### **Sample size**

How many cases a result is based on. Small samples support small claims.

*In our project:* 30 questions, 3 people, 1 model.

### **Median latency**

The middle value of how long answers took. Shown in the summary but not used in the paper, since we make no timing claims.

## 11.5 Tax and money words

### Financial year, assessment year

The financial year is when income is earned (April to March). The assessment year is the next year, when it is taxed.

*In our project:* Our rules: financial year 2025-26, assessment year 2026-27.

### Old regime, new regime

Two sets of tax rules a person chooses between. The new regime has lower rates but almost no deductions; the old regime has higher rates and many deductions.

### Slab

A band of income taxed at its own rate. Only the part of your income inside a band is taxed at that band's rate.

### Standard deduction

A fixed amount knocked off salary with no proof needed: ₹75,000 in the new regime, ₹50,000 in the old.

### Taxable income

Income after everything allowed has been subtracted. The slabs apply to this.

### HRA exemption

The part of a house rent allowance that is not taxed: the smallest of three amounts (HRA received; 40 or 50 percent of basic salary; rent paid minus 10 percent of basic). Old regime only.

### Section 80C, 80D

80C allows up to ₹1,50,000 of certain investments (PPF, ELSS, life insurance) to be deducted. 80D covers health insurance. Old regime only.

### Section 87A rebate

Cancels tax completely below an income limit. In the new regime for 2025-26, tax is zero up to ₹12,00,000 of taxable income.

*In our project:* The new regime's "marginal relief" just above that limit is not implemented.

### Cess

An extra 4 percent charged on the tax.

### TDS, section 194J

Tax deducted at source: tax the payer holds back before paying you. Under 194J, Indian businesses deduct 10 percent from professional fees. Overseas clients deduct nothing.

### GST

Goods and Services Tax, charged on sales and passed to the government.

### Input tax credit

GST you paid on business purchases, which you can subtract from the GST you charged.

### GST registration threshold

The turnover above which registering for GST is compulsory: ₹20,00,000 for services here. Arjun is above it; Rohan is ₹2,00,000 below.

### Zero-rated

Exports of services: no GST charged, but credit can still be claimed.

### Presumptive taxation, 44AD, 44ADA

Declaring a fixed share of turnover as profit instead of keeping full accounts. 44AD for businesses: 6 percent of digital receipts plus 8 percent of the rest. 44ADA for professionals: 50 percent of receipts.

### Advance tax

Paying tax in instalments during the year (June, September, December, March) instead of all at the end.

## **Form 16, ITR**

Form 16 is the certificate an employer gives showing salary and tax deducted. An ITR is the income tax return filed with the department.

## **Income-tax Act, 1961 and Income Tax Act, 2025**

The 1961 Act governs our year, 2025-26. The 2025 Act took effect on 1 April 2026 with a new section structure. Moving to it means a new rulebook.

## **11.6 Research and paper words**

### **Conference, proceedings**

A conference is an event where researchers present papers. Proceedings are the collected papers, published or handed to attendees.

*In our project:* BAICONF 2026 at IIM Bangalore.

### **Abstract**

The short summary at the top of a paper. Conferences often accept or reject on the abstract first.

*In our project:* Ours was accepted; one sentence in it needs correcting with the guide's approval.

### **Full paper**

The complete write-up submitted after the abstract is accepted.

*In our project:* Due 31 October 2026, at most 4,500 words.

### **Peer review, double-blind**

Other researchers judge a paper before it is accepted. Double-blind means neither side knows the other's identity.

*In our project:* BAICONF does not say its review is blind.

### **Literature review**

The part of a paper that explains what others have already done and where your work fits.

### **Reference, citation**

A reference is a full entry in the list at the end of a paper. A citation is the pointer to it in the text, like [7].

### **Source table**

Our checked list of every reference: full citation, link, how it was verified, and what the paper uses it for.

*In our project:* paper/Source\_Table.docx.

### **DOI**

A permanent ID for a published paper, a code starting with "10.", that always leads to it.

### **arXiv, preprint**

arXiv is a free website where researchers post papers, often before formal review. A preprint is a paper posted there that may not have been reviewed. Where a reviewed version exists, we cite that.

### **Verified reference**

One we actually found and read (at least the abstract), with every detail of the citation checked.

Three from the old list could not be found anywhere and were dropped; two had the wrong authors and were corrected.

### **Novelty, contribution**

What a paper adds that did not exist before. Stated precisely, it is our strongest selling point; stated loosely, it is the easiest thing for a reviewer to knock down.

### **Claim**

Anything the paper states as true. Every claim must be backed by our results or a cited source.

*In our project:* Master plan section 17 lists what we may and may not claim.

**Limitation**

Something the work does not do or does not show, stated openly. Reviewers trust a paper more when it names its own limits.

**Methodology, findings**

Methodology is how the system was built and tested. Findings are the results.

**Technology Readiness Level (TRL)**

A 1 to 9 scale of how mature a technology is. Level 4 means it works in a laboratory on representative data; level 9 is proven in real use.

*In our project:* We are at level 4.

**NCFE**

The National Centre for Financial Education, set up by India's financial regulators. Its 2019 survey found 27 percent of Indian adults financially literate.

**BAICONF, ICADCML**

BAICONF is the IIM Bangalore conference we are going to. ICADCML was an earlier target (a Springer conference) that we dropped.